

Month Prediction - Highest Road Accidents Probability

1 Team Members:	Shaleen Agarwal	Piter Zacari Garcia Bautista	Lohit Velagapudi
Emails:	sa4421@rit.edu	pzg8794@rit.edu	lv4847@rit.edu

2 Overview

The dataset used for the purposed of this project is the Road Safety Data (Data.gov.uk, 2014), which was obtained from the website of the government of the United Kingdom. This dataset has the data of the road accidents that happened in the United Kingdom from the year of 2005 to 2013. We have only chosen all road accidents that occurred in the city of Birmingham during the same time period. The link to access this database was added to the reference section of this project along with a reference to our dataset (Drive.Google.com, 2014). Our dataset is a modified version of the Road Safety Data dataset containing only the road accidents of the city of Birmingham. After data preparation, our dataset has 18,937 instances and 22 attributes containing only information about unique road accidents and most injured drivers involved. The objective of this project is to achieve the data mining goals pertaining to our hypotheses through the CRISP-DM steps implementation. The CRISP-DM steps have been summarized in the Hypotheses, Data Preparation, Data Mining, and Comprehensive Analysis sections of this project. First, the hypotheses specifies the goal of our project, and the reason to why we consider our hypotheses important topics for our project. Second, the data preparation explains the approach we utilized to properly transform, format and clean our dataset to suit our classification models and data mining goals. Third, the data mining describes the algorithms and classification models utilized in our project. It gives a brief comparison about the different classification models used in our data, and the reasons to why we have chosen these algorithms to achieve our data mining goals. Lastly, comprehensive analysis specifies the general approaches and decisions taken throughout the entire project development to successfully achieve our goals.

3 Hypothesis

The first hypothesis of this project is to use data mining techniques to build a classification model to predict the months in which road accidents are more likely to happen. The reason why we felt that this problem was important is because our team anticipated that the months with more vacation days and holidays will most likely have a higher rate of road accidents. Hence, we have extracted the months from the date attribute and made months our classifier to set our data mining goal to predict the month in which more accidents are likely to occur. The second hypothesis of this project is to use data mining techniques to build a classification model and predict the age-

groups that are most likely to cause road accidents. The reason why we assume this to be important is because certain age groups could be at higher risk than others in certain locations. Therefore, to classify our dataset into age groups we created another attribute called CAgeGroup. We used the age-band provided by our dataset to assign the corresponding age group to each driver. This CAgeGroup is a categorical classifier and it only contains nominal values such Youth, Early-Adult, Adult, Late-Adult, Early-Senior, and Senior, based on this classifier we were able to state our second hypothesis. In brief, we have deployed different classification models and compared them. After the comparison, testing and validation process we have chosen the mode that met our main hypothesis.

4 Data Preparation

First Attempt

In the first attempt our dataset had five years of road accidents reported in the city of Leeds, England. We had a total of 16,381 instances, but this dataset failed every algorithm built to prepare and clean our dataset. These algorithms are described in the data mining section, below are the problems that we faced and the solutions taken to overcome them.

Problems: First, the dataset had many invalid, unknown and unclassified values. This would not be an issue for us to resolve if the following problems did not occur. Second, after running the dataset through a Java algorithm built to find unique records, the dataset was reduced from 16,381 to 7,922 instances. We lost more than 50% of the data only in records with the same id. Third, after running the dataset through another java algorithm built specifically to find unique instances, the dataset content was reduced from 7922 to 6684. As we mentioned above we were only interested into drivers records, but not multiple records from different drivers about the same accident.

Solutions: First, the dataset did not meet the requirements, therefore we had to change to another dataset. However, due to all the work and analysis done on this dataset we opted to look for a similar dataset to be able to implement the algorithms we had already built. We obtained a database that contains road traffic accidents of multiple cities in Great Britain including Leeds. This dataset contains data from 1979 to 2013, more data than what we could handle with but this was our only

solution. Second, to avoid running through the same issue, we decided not to use the same city and queried the data from this huge database to find the city that has more number of accidents reported. In order for us to do this, we explored the data by using R and obtained Birmingham as the city with most accidents records.

Second Attempt

To obtain our current dataset we queried the Road Safety Data datasets, merged resulting datasets, and further integrated the merged datasets.

Query Road Safety Data Datasets:

Each dataset was queried to find only road accidents records of the Birmingham city. There was 24 huge datasets in total; 8 datasets for accidents, 8 datasets for vehicles, and 8 datasets for casualties. The accident dataset is more than 200 GB, casualty dataset is more than 83 GB, and vehicle dataset is more than 160 GB. It was not possible to load any of these huge datasets in a program and simply query them, therefore a code was created specifically for this step.

Merging Datasets

The results of the steps performed in the query section were 8 datasets containing only records for the accidents, vehicles, and casualties occurred in the city of Birmingham. The following step was to merge all eight datasets into one dataset for the casualties, vehicles, and accidents records. This was achieved by modifying the algorithm used in the Query.

Further Data Integration

The data integration was done after some of the cleaning steps were performed. This is explained below, in the Data Cleaning section, and it was done this way to avoid doing a lot of work on a bad dataset.

Data Cleaning

To avoid running into the same problems we did in the first dataset, we decided to first remove duplicate instances in our datasets, then remove common records, convert non-numeric attributes to nominal, map all invalid data to a single value, integrate datasets, and handle invalid data.

Duplicate Records Removal:

All accidents records obtained from the queried section were unique records and each one represents only one accident occurrence and not 2 or 3 as our previous dataset. Each record has a unique index that links them to the vehicle and casualty datasets; these datasets had duplicated records, but we decided to develop a plan to delete these duplicated records during the data integration step to avoid losing relevant information.

Common Records Removal:

In order to find records that were not unique, the dataset was ran through an algorithm built exactly to find unique accidents in the accidents dataset. This algorithm helped us to avoid having two drivers with different report id, but reporting the same accident. To achieve this, we chose a combination of attributes that define a road accident.

Using the algorithm mentioned above we were able to find unique records. Since we only needed drivers, another algorithm was built to find all unique instances in the casualty dataset and then use the id of these records to find them in the accidents dataset. We first had 28,553 records but after running the dataset through the algorithm mentioned above we had 18,937 instances. Once again, we ran this dataset through the algorithm we built to find unique accidents. All driver records happened to be unique accidents, this is because the accidents dataset does not contain any information about the casualty or driver but only information about the road accident.

Data Conversion:

Our dataset was not partially converted into nominal as the previous dataset. Therefore, we had to convert all attributes (except numeric attributes such longitude, latitude, among others) in the entire accidents, vehicle, and casualty datasets to nominal. This step had the following consequences in the development of our project:

Consequences: The accident dataset had 35 attributes from which 19 were nominal, the vehicle dataset had 24 nominal attributes, and the casualty dataset had 15 nominal attributes. We did not want to remove attributes, except for the local authority attributes that were the number of the city, until we were completely sure that a specific attribute would not be useful in our dataset. On the other hand, converting the entire dataset to nominal gave us the ability to control the size and labeling of each attribute value. This made it easier for us to manage our data after mapping it as explained in the mapping section. More importantly, our classifier is categorical and representing the attributes in nominal form made it much easier for us to understand and analyze our dataset.

Data Mapping:

During the data conversion, each value was mapped to easily identify dirty and noisy data. Since it is possible that during the data convention noise can be inserted, the following techniques were implemented:

1. Value size: Each attribute value has a unique size, this way it is easier, less time and memory consuming when comparing records.
2. Invalid data: Since every attribute value has a unique size

so does the invalid data. All empty or out of range values were mapped to the string Invalid. This would make it easier for us to handle all Invalid data with the appropriate tools.

3. Noise: To test whether or not noise was input during the data conversion, a Case with If statements was used. Any attribute value that was not converted to its nominal value appeared as NA, decreasing the amount of troubleshooting during the cleaning process. It was easy, after data conversion, to search for NA values. If any NA were found then the attribute where this appeared had to be fixed. This was a very iterative procedure, but it was a secure way to convert the data without inputting noise.

Data Integration:

This was one of the most difficult steps, after the invalid data handling, during the cleaning process. The process was to delete records that were not unique, and to keep only the information that we consider relevant for our project. For this step there were two approaches, and the one with the best results was chosen.

1. First Approach:

An algorithm using map-hashing function was built to replace duplicate records for the records with most repetition. We believed that keeping a record with most repeating patterns would help our model to detect unique patterns in a car accident. However, this was not the case with the map-hashing algorithm since we had issues with drivers containing data not pertaining to them.

2. Second Approach:

We decided to let the data decide, in terms of priority, which record contains most important information about the road accident in the casualty dataset. Then, we linked to this record the matching record from the vehicle dataset.

Casualty Severity: The casualty dataset contains an attribute named *Casualty – Severity*. This attribute measures the severity level of how hurt a driver was in a road accident. Instead of map-hashing the values, we decided to select the records from drivers whose severity level was higher.

Vehicle Record: After obtaining the record with highest casualty severity level, we looked for this record in the vehicle record and merge them together.

Record Integration: At this point we have already built a record for the vehicle and casualty resulting record. This record is then integrated with the accident record that it belongs to.

After applying the algorithm mentioned above, we obtain a single dataset containing information about the accident,

vehicle, and casualty datasets; three datasets with unique records that define unique road accidents.

Handling Invalid Data:

The attributes that contained most of the invalid data were two of the most important attributes in our dataset, the age and the road class attributes. After the preparation, cleaning algorithms, and techniques mentioned above we only had the following two choices to do to handle the data. First, we could use Invalid as a value type for both attributes and explain the meaning of this in our report. Second, use data mining algorithms to predict the invalid data based on the behavior of the attribute values that relate to this attribute. For this option, a hybrid system was built to predict the ages and road classes missing in the attributes. This hybrid model is better explained in subsection Method 1 of the Data Mining section.

5 Data Mining Methods

To achieve our data mining goals we implemented multiple methods to better clean our dataset, create classification models, compare and choose the right model for the goal of our project.

1. Method 1 - Hybrid System:

This hybrid system consists of three different data mining techniques such as clustering, classification, and data imputation.

Clustering: Multiple clusters of records are assigned to a record that has an attribute with a missing value.

Cluster Selection: These clusters are selected by analyzing specific attributes values of each record in the accidents dataset. In the data conversion step every nominal value was assigned a unique size, this makes clustering easier since now we can use the sum of all attributes sizes as the distance of other records to the record with the missing value.

Classification: After assigning the clusters, the records are classified into classes that are more meaningful to the record with the missing value. The classes are selected by analyzing the data contained in each clustered record.

Classes Selection: when predicting the missing age-band of a record, after clustering all records based on their attribute weight (distance), the clustered records would be classified into class A, B, C, D, and O. Class A contains records with vehicles that can be driven only by drivers that are 16+, class B contains records with vehicles that can be driven only by drivers that are 21+, class C contains records with vehicles that can be driven only by drivers that are 24+, and class D contains records with vehicles that can be driven only by drivers 4+. It is important to

understand what this means, and check the vehicle type of the record with the missing age. Also, it is important to know what age-group the clustered record belongs to before classification. If a missing value record falls within class A, B, or C, every record whose age group is teenager must be disregarded since teenagers cannot drive these types of vehicles.

After classifying all records into classes A, B, C, D, and O. The size of each list is used for analysis to obtain the mode of the size of each list in a big list. Once this size is found it is used to select the class that it corresponds to, if two lists happened to have the same size the one containing the vehicle type of the missing record is chosen. Otherwise, if the missing value record vehicle class does not match, the one with the highest age range is chosen. After obtaining all records of one class, these records are classified into age-group and once again the size of the lists are used to find the mode-size. This size is used to find the age-group to which it belongs to, and then the age-group and age-range of the first record in the list is assigned to the missing record.

2. Method 2 - Hybrid System:

The same approach used to predict the age-bad of record was used to predict the road-class values that were missing. In the cluster selection, all clusters were assigned to the missing value record by calculating the distance based on the unique weights assigned to each nominal value. After obtaining the clustered records, these were classified into lists with a unique speed limit such as 10, 20, 30, 40, 50, and 60. Then, the same approach as in the age attribute was used to obtain the mode-size based on the size of each list. If two lists had the same size, the one with the speed limit of the missing record was chosen. Otherwise, if the speed limit of the missing value record did not match any record, the one with the highest speed limit was chosen. Then, after obtaining the list of records, the obtained list is classified into road types and same mode-size imputation methodology is used to find a list of records that are most similar to the missing value record. Then, the record class gets replaced with the road class of these records.

3. Data Modeling:

We have used our dataset to create multiple classification models with different classification algorithms. The classification algorithms that we used to model our data are Naive Bayes, Random Forest, J48, and BFTree. We chose these algorithms because they were able to perform the data analysis better and faster than other algorithms, and they were proven to work very well with nominal attributes. In the development of each one of these models we realized that their response was different when split-

ting the data 50% for training and 50% for testing to when splitting the data 80% for training and 20% testing.

Creating Model:

Splitting - 50/50: Using Month as our classifier.

In this test, classifications algorithms such as Nave Bayes and J48 failed to build a good model in the training and testing phase of our model. The only classification algorithm that gave a good result when training was the Random Forest. However, this classification algorithm failed to correctly classified most of the instances in the testing set.

Splitting -80/20: Using Month as our classifier.

Dataset with 22 attributes - Classification Algorithms						
Instances	Naïve Bayes		Random Forest		J48	
	Train	Test	Train	Test	Train	Test
Incorrect	2258	3300	30	3394	6053	3400
Correct	12888	490	14994	396	9093	390

In this test, the Nave Bayes, Random Forest, and the J48 classification results improved. It is very clear that in order for us to better test our dataset we need to split our dataset into 80% training and 20% testing. Even though all classifiers algorithms failed to correctly classify most instances during the testing phase of our model, we discovered the proper data percentage to use when splitting our dataset into a training and testing set. Also, we realized that removing attributes to improve the results did not help. We decided to test other attributes as a classifier such as the age-group, in case none of the models created above achieved our goal.

Splitting - 50/50: Using Age-Group as our classifier.

In this test, classifications algorithms such as Nave Bayes and Random Forest correctly classified most instances where as J48 correctly classified all instances.

Splitting -80/20: Using Age-Group as our classifier.

In this test, classifications algorithms such as Nave Bayes and J48 correctly classified all instances during the testing phase. Even though we got very good results with age-group as the classifier, it does not mean our model is correct. The only way for us to know if the model is correct is by testing our model, one more time, with a real case scenario such as providing the data for 2013 for prediction since the testing model was built with data from 2005-2012. Based on the results we obtained from test performed, we have obtained multiple models. However, we are not sure which one is correct or suits our hypothesis. Therefore, we decided to test our models by feeding them all data from the year of 2013 to see if any of them would accurately determine the month in which most accidents happened. If any of the models predicts the right month, then this would be the model that suits our hypothesis.

Model Testing - Splitting -80/20:

Dataset with 22 attributes						
Classification	Model 1		Model2		Model3	
Instances	Naïve Bayes		Random Forest		J48	
	Model	2013	Model	2013	Model	2013
Incorrect	2258	218	30	270	6053	791
Correct	12888	1405	14994	1353	9093	832

When testing our models with 14 attributes, all models failed to predict the right month. As we emphasized during the model creation phase, we learned that reducing the number of attributes to 14 did not improve our results. Therefore, we preceded to test our model with 22 attributes. As a result, Nave Bayes and J48 failed to predict the right month. However, our Random Forest model successfully predicted the month in which most accidents happened during the year of 2013.

Model Validation:

By exploring the road accidents data of the year of 2013, we were able to validate the correctness of our model. The month in which most accidents occurred is the month of October with 179 accidents reported, based on the description of the data of the year of 2013 generated in R.

	Jan	Feb	Apr	May	Nov	Jun	Dec	Mar	Jul	Sep	Oct	Aug
Jan	92	2	0	1	4	0	2	1	0	0	4	1
Feb	1	81	1	2	1	2	1	2	2	1	0	1
Apr	1	3	105	1	1	9	0	3	2	2	4	2
May	0	2	1	132	2	1	3	4	3	3	1	0
Nov	3	2	0	4	147	1	4	0	2	2	9	2
Jun	0	3	2	3	1	115	0	1	2	5	1	2
Dec	2	0	2	1	1	1	87	1	1	4	0	4
Mar	2	3	3	4	3	1	7	92	1	2	0	0
Jul	3	3	4	3	4	3	2	3	133	1	3	2
Sep	0	0	6	2	3	4	2	2	4	108	3	1
Oct	5	2	4	4	4	3	3	3	6	1	140	4
Aug	5	0	5	5	0	4	0	2	2	2	1	121

This is the confusion matrix generated by our model built with the Random-Forest classification algorithm. It tells us that most accidents occurred in the month of October, as you can see on the top right of the matrix. Based on the Model Testing and Validation phases of this project, the Random-Forest classification algorithm was the only classifier (out of the three classifiers compared above) that created a model suitable for our project.

6 Comprehensive Analysis

During the implementation of this project our team learned important lessons to keep when analyzing big data. Some of these lessons were learned during the data understanding, data cleaning, and data modeling of our project.

Data Understanding Analysis: We learned that the most important step in the CRISP-DM is the data understanding. In our first dataset, we did not have any source of information explaining the usefulness of each attribute, how the dataset was used before, and where exactly it came from. This led to

misleading assumptions made by our team in terms of what the attributes really meant, and the importance of these in our data and goal of our project. We found ourselves in a constant loop, going from the cleaning phase to the data understanding phase constantly. Finally, we realized that the dataset did not meet the requirements of our project, and it was not good enough for our goal because it had too many duplicated and common records.

On the other hand, when we changed our dataset to a much bigger dataset with a similar format. We were able to obtain a complete description of the dataset such as attributes meaning, where it comes from, and usage of the dataset. Unfortunately, the dataset had not been used before in big data analysis. This is understandable because it is a very big dataset, which gave us a lot work to query and to built our dataset from the original data. Once we studied the attributes descriptions and usage of the dataset, we were able to move quickly to modify and re-use all algorithms previously built to prepare and clean our old dataset.

Data Cleaning Analysis:

We initially divided our cleaning phase into three stages to distribute the work load, understand and compare our work.

First Dataset:

In the stage one, we integrated our data by using a Linux command, transformed data from numeric-nominal to nominal using a simple Java code, summarized the data and created new features such as the attributes age-group, and month (our classifier.) In the stage two, we explore the data using tools such as Weka and R to find possible outliers and dirty data within our dataset, and prune our dataset if necessary. In the third stage, we were not able to move forward because when implementing our cleaning algorithms to remove duplicate records we ended up with less than 7000 instances. This led our group to quickly find a new dataset with a structure similar enough to match our hypotheses.

Second Dataset:

In the second dataset, we mainly focused on understanding the data. At this point, we had realized that our biggest mistake was not knowing what our dataset really meant. For example, our first dataset was sorted based on casualty class such as driver, passenger, and pedestrian and not on accidents. Therefore, we had cases in which three different reports from three different casualty class were referring to the same accident. We had to avoid this, and the only way was by understanding what our dataset really says. Fortunately, we discovered that the dataset we queried and built from the huge dataset was sorted by accidents, and that the casualties and vehicles information about the accident was provided in two different datasets. This made our work easier in terms of finding unique records but, it

did not make our cleaning and data mining phases any easier as you can tell from reading the data preparation and data mining sections in this report. For the invalid data of our dataset, we decided to use a hybrid system based on algorithms we have learned in this class. Even though clustering, classification, and data imputation algorithms are simple methods, they proved to be a very powerful tool when combined to learn and predict invalid data in our dataset.

Data Modeling:

In the process of creating our models we decided to follow the advice of not focusing too much in the percentage of the correctness of our model. Therefore, we decided to create our models, compare models, test and validate our models without disregarding any model based on the percentage of correctness.

Models Comparison:

When creating our models we realized that the model with most percentage of correctness was J48. However, we did not disregard the other models but proceeded to test each one. To do so, we split our dataset into training and testing sets. Since we did not know which partition would be appropriate for our dataset we tried splitting our dataset into 50%/50%(50% training and 50% testing) and 80%/20%(80% training and 20% testing). In every test, 80%/20% resulted to be the right percentage to split our dataset for testing purpose, we also try other percentages but they did not make any difference. After testing our datasets, we did not have very good results from each one of the classifiers. The only classifier that gave better results was the Random Forest, but it was not very promising for our project hypothesis so we decided to test other attributes such as age-group to see its models behaviors. All classifiers gave good results, but we decided to validate our model before making any further assumptions.

Model Validation:

In this phase is when we test if any of the first models suits our main hypothesis. The validation process was based on imputing our model a new dataset containing only data from the year of 2013, as it was mentioned in the data modeling subsection, we used the data from 2005-2012 to build and test our models and inputting this new dataset into our models would be a real scenario test. The only model that correctly classified most instances was the Random Forest, and therefore this is the right model for our data using the attribute Month. The only step left was to check if our model proves our hypothesis.

Proving Hypothesis:

As expected, our model proved our hypothesis. Based on the matrix confusion generated by our model, it predicted that most accidents are likely to occur in October. In order to prove this,

we proceeded to explore the dataset containing only data from the year of 2013 in R. R results showed that most accidents actually occurred in October, proving that our model has correctly predicted the right month. It is important to highlight that our model predicts the month in which most accidents are likely to occur, but it does not accurately predicts the number of accidents occurred in this month.

7 Lessons Learned & Discussion

In the process of implementing the CRISP-DM steps to achieve our project and data mining goals we learned very important lessons. This lesson has taught us to look at data and approach the data analysis in a different way, especially when we are dealing big data. Some of these lessons can be explain through the issues we faced in the data understanding, data cleaning, and data mining phases of our project.

First, it does not matter how long it takes to understand your data. It is important to know what the data says, where it comes from, and to find out whether or not it has been used before. If the dataset has not been used before, looking up the description of each attribute and what it means in the scope of your project will help you to make better assumptions.

Second, we learned that the first step before cleaning a dataset is to find duplicates. In this process, you must clearly understand your data and each attribute. This way, you will be able to create your own algorithm to find instances that share the same values even though they have a unique id.

Lastly, we learned that validating your model is the best way to determine whether or not your model is working. The percentage of correctness of your model may not be high, but it does not mean that your model is bad or good. You must validate your model to test if the model suits your hypothesis. If the model does not pass the validation test, then you may make further assumptions about your model or data.

8 References

- [1]Data.gov.uk, (2014). Road Safety Data - Datasets — data.gov.uk. [online] Available at: <http://data.gov.uk/dataset/road-accidents-safety-data> [Accessed 16 Nov. 2014].
- [2] <https://support.google.com/mapmaker/answer/1098048?hl=en>
- [3] Data.gov.uk, (2014). Road Safety Data - Datasets data.gov.uk [online] Available at: <http://data.gov.uk/dataset/road-accidents-safety-data> [Accessed 16 Nov. 2014].
- [4]Drive.google.com, (2014). Google Drive - Shared Project Datasets, & Data Auxiliary Docs, & More. [online][Accessed 16 Nov. 2014] Available at: <https://drive.google.com/folderview?id=0B6-Wz8uQfNpHUzVGTS1OcEJCvk0&usp=sharing>